

个人答辩报告

12312505 江博成

SUSTC

May 14, 2025

负责工作简述

Controller
○○○○○○○○

负责工作简述

- 1 顶层架构与接口设计
- 2 Controller 控制信号生成模块
- 3 Decoder 立即数生成与寄存器模块
- 4 WriteBackMUX 写回 MUX 模块
- 5 基本测试场景 1,2

负责工作简述
○

Controller
○○○○○○○○

Content

1 负责工作简述

2 Controller

12312505 江博成

个人答辩报告

May 14, 2025

2 / 11

负责工作简述
○

Controller
●○○○○○○○

Controller

```
1 module Controller (  
2     input  [31:0] inst,  
3     output reg    branch,  
4     output reg [1:0] alu_op,  
5     output reg    alu_src,  
6     output reg    mem_read,  
7     output reg    mem_write,  
8     output reg    mem_to_reg,  
9     output reg    reg_write,  
10    output reg    is_jal,  
11    output reg    is_jalr,  
12    output reg    is_lui,  
13    output reg    is_auipc,  
14    output reg    is_ecall,  
15    output reg    en_pc,  
16    output reg    en_input,  
17    output reg    en_output,  
18    input  [31:0] reg_a7, // ecall  
19    input          need_to_check  
20    input          done_input // let  
21    en_pc = 1  
22 );
```

```
1 // Opcode definitions (parameterized  
2     parameter R_TYPE = 7'b0110011;  
3     parameter I_TYPE1 = 7'b0010011;  
4         // addi  
5     parameter I_TYPE2 = 7'b0000011;  
6         // lw  
7     parameter I_TYPE3 = 7'b1100111;  
8         // jalr  
9     parameter I_TYPE4 = 7'b1110011;  
10        // ecall  
11    parameter S_TYPE = 7'b0100011;  
12    parameter B_TYPE = 7'b1100011;  
13    parameter U_TYPE1 = 7'b0110111;  
14        // lui  
15    parameter U_TYPE2 = 7'b0010111;  
16        // auipc  
17    parameter J_TYPE = 7'b1101111;  
18        // jal  
19    wire [6:0] opcode = inst[6:0];
```

Controller

```
1 always @(*) begin
2     // Default all outputs
3     branch      = 0;
4     alu_op       = 2'b11;
5     alu_src      = 0;
6     mem_read     = 0;
7     mem_write    = 0;
8     mem_to_reg   = 0;
9     reg_write    = 0;
10    is_jal       = 0;
11    is_jalr      = 0;
12    is_lui       = 0;
13    is_auipc     = 0;
14    is_ecall     = 0;
15    en_input     = 0;
16    en_output    = 0;
17    en_pc        = 1;
18    case (opcode)
19        R_TYPE: begin
20            alu_op    = 2'b10;
21            alu_src    = 0;
22            reg_write = 1;
23        end
```

```
1 I_TYPE1: begin
2     alu_op    = 2'b11; // check here
3     alu_src    = 1;
4     reg_write = 1;
5 end
6 I_TYPE2: begin // lw
7     alu_op    = 2'b00;
8     alu_src    = 1;
9     mem_read  = 1;
10    mem_to_reg = 1;
11    reg_write  = 1;
12 end
13 I_TYPE3: begin // jalr
14     alu_op    = 2'b11;
15     alu_src    = 1;
16     reg_write = 1;
17     is_jalr   = 1;
18 end
19 S_TYPE: begin // sw
20     alu_op    = 2'b00;
21     alu_src    = 1;
22     mem_write = 1;
23 end
```

Controller

```
1 I_TYPE4: begin // ecall
2     is_ecall = 1;
3     case (reg_a7)
4         1, 34, 35: begin
5             en_output=1;
6         end
7         5: begin
8             reg_write=1;
9             en_input=1;
10            if (done_input) en_pc=1;
11            else en_pc=0;
12        end
13        default begin
14        end
15    endcase
16 end
17 B_TYPE: begin // beq, bne, etc.
18     alu_op    = 2'b01;
19     branch    = 1;
20 end
```

```
1 U_TYPE1: begin // lui
2     alu_op    = 2'b11;
3     alu_src    = 1;
4     reg_write = 1;
5     is_lui     = 1;
6 end
7 U_TYPE2: begin // auipc
8     alu_op    = 2'b11;
9     alu_src    = 1;
10    reg_write  = 1;
11    is_auipc   = 1;
12 end
13 J_TYPE: begin // jal
14     alu_op    = 2'b11; // don't
15         care
16     alu_src    = 1; // don't care
17     reg_write  = 1;
18     is_jal     = 1;
19 end
20 default: begin
21     en_pc      = 0;
22     // All control signals remain
23     default
24 end
```

Testbench of Controller

```
1 module tb_Controller;
2
3     // inputs
4     reg [31:0] inst;
5     reg [31:0] reg_a7;
6     reg        done_input;
7
8     // outputs
9     wire        branch;
10    wire [1:0]    alu_op;
11    wire        alu_src;
12    wire        mem_read;
13    wire        mem_write;
14    wire        mem_to_reg;
15    wire        reg_write;
16    wire        is_jal;
17    wire        is_jalr;
18    wire        is_lui;
19    wire        is_auipc;
20    wire        is_ecall;
21    wire        en_pc;
22    wire        en_input;
23    wire        en_output;
```

```
1 Controller uut (
2     .inst      (inst),
3     .branch    (branch),
4     .alu_op    (alu_op),
5     .alu_src   (alu_src),
6     .mem_read  (mem_read),
7     .mem_write (mem_write),
8     .mem_to_reg(mem_to_reg),
9     .reg_write (reg_write),
10    .is_jal     (is_jal),
11    .is_jalr   (is_jalr),
12    .is_lui    (is_lui),
13    .is_auipc  (is_auipc),
14    .is_ecall  (is_ecall),
15    .en_pc     (en_pc),
16    .en_input  (en_input),
17    .en_output (en_output),
18    .reg_a7    (reg_a7),
19    .done_input(done_input)
20 );
21 initial begin
```

Testbench of Controller

```
1 // dump waveform
2 $dumpfile("tb_Controller.vcd");
3 $dumpvars(0, tb_Controller);
4 // ----- Test R_TYPE -----
5 inst = {25'd0, 7'b0110011}; //
6     R_TYPE opcode
7 #1;
8 if (alu_op != 2'b10 || reg_write
9     != 1) $fatal("R_TYPE failed");
10 // ----- Test ADDI (I_TYPE1) -----
11 inst = {20'd0, 5'd1, 3'b000, 5'd2,
12     7'b0010011};
13 #1;
14 if (alu_src != 1 || alu_op != 2'
15     b11 || reg_write != 1) $fatal(
16     "ADDI failed");
17 // ----- Test LW (I_TYPE2) -----
18 inst = {20'd0, 5'd3, 3'b010, 5'd4,
19     7'b0000011};
20 #1;
21 if (!(mem_read && mem_to_reg &&
22     reg_write && alu_src==1 &&
23     alu_op==2'b00))
24     $fatal("LW failed");
```

```
1 // ----- Test ECALL input done -----
2 done_input = 1;
3 #1;
4 if (en_pc!=1) $fatal("ECALL input
5     done failed");
6 // ----- Test SW (S_TYPE) -----
7 inst = {20'd0, 5'd7, 5'd8, 3'b010,
8     5'd0, 7'b0100011};
9 #1;
10 if (!(mem_write && alu_src && alu_op
11     ==2'b00)) $fatal("SW failed");
12 // ----- Test BEQ (B_TYPE) -----
13 inst = {7'b0000000, 5'd0, 5'd0, 3'
14     b000, 4'b0000, 1'b0, 7'b1100011
15     };
16 #1;
17 if (!(branch && alu_op==2'b01))
18     $fatal("BEQ failed");
19 // ----- Test LUI (U_TYPE1) -----
20 inst = {20'hABCD0, 5'd0, 7'b0110111
21     };
22 #1;
23 if (!(is_lui && reg_write && alu_src
24     )) $fatal("LUI failed");
```

Testbench of Controller

```
1 // ----- Test AUIPC (U_TYPE2)
2 inst = {20'h10000, 5'd0, 7'b00101111};
3 #1;
4 if (!(is_auipc && reg_write &&
5 alu_src)) $fatal("AUIPC
6 failed");
7 // ----- Test JAL (J_TYPE) -----
8 inst = {20'd0,5'd0,3'b000,5'd0
9 ,7'b11011111};
10 #1;
11 if (!(is_jal && reg_write))
12 $fatal("JAL failed");
13 $display("All Controller tests
14 passed");
15 #1; $finish;
16 end
17 endmodule
```

```
1 // 说明：Controller模块主要接受的
2 input的inst中的opcode,
3 testbench生成了具有不同opcode的
4 inst作为Controller模块的输入。
5 2 // 对于课上没有提到如何实现的指令，
6 我们直接增加了is开头的控制信号，
7 来做额外的分类讨论。
8 3 // 对于输入输出，我们是直接实现的
9 ecall，也就需要额外的输入信号
10 reg_a7。
11 4 // 若reg_a7 为1,34,35，则对应输出，
12 令使能信号en_output=1，output模
13 块接受该信号执行输出判断。
14 5 // 若reg_a7 为5，则对应输入，因为输入
15 需要等待确认信号，不能在一个
16 时钟周期内完成，故令使能信号
17 en_pc=0，各模块接受该信号中断时
18 序，并令en_input=1，传给input模
19 块并参与后续逻辑，等待input模块
20 传回done_input型号，令en_pc=1重
21 新启动时序。
22 6 // 以上逻辑测试在testbench代码中均有
23 显现，波形图也符合设想逻辑。
24 7 // 最终Tcl console输出为：All
25 Controller tests passed
```

Waveform of Controller

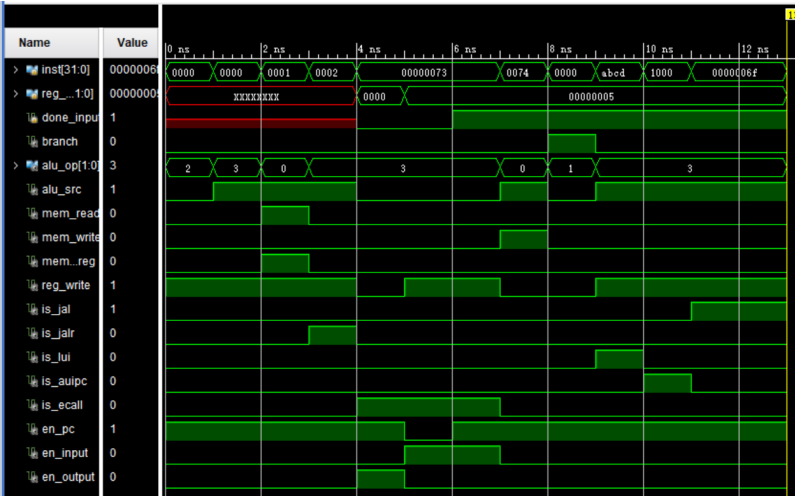


Figure: Waveform of Controller